

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 7. String and its Methods

Lecture: 75. Joining and Splitting a String

Section 1: Lecture Summary

This lecture covers several important string methods in Python: `replace()`, `join()`, `split()`, `rsplit()`, and `splitlines()`. The instructor explains how each method works conceptually and demonstrates practical examples in a Jupyter Notebook environment. The `replace()` method is used to substitute old substrings with new ones, optionally limiting the number of replacements. The `join()` method inserts a given separator string between all elements of an iterable (typically a string or list of strings) to create a new concatenated string. The `split()` method divides a string into a list of substrings based on a specified separator and supports limiting the number of splits. The `rsplit()` method is similar but splits starting from the right side. The `splitlines()` method handles splitting multi-line strings into individual lines. The lecture emphasizes that strings are immutable; hence, these methods return new strings or lists without modifying the original string. Students are encouraged to practice by trying various examples to solidify their understanding.

Section 2: Key Concepts and Explanations

- String Immutability

- Strings cannot be changed once created.
- Methods that modify strings return new string objects.

- `replace(old, new, count)`

- Replaces occurrences of the substring ``old`` with ``new``.
- ``count`` is optional; limits how many occurrences to replace.
- If ``old`` substring is not found, returns the original string unchanged.

- Example: Replace hyphens with commas.
- Useful for text manipulation and format changes.

- **join(iterable)**

- Inserts the string on which join() is called between elements of an iterable (usually a list or string).
- Returns a new concatenated string.
- Does not add the separator after the last element.
- Example: ``/'` .join('ABC')`` results in ``A/B/C``.
- Useful for constructing strings from lists or sequences.

- **split(sep=None, maxsplit=-1)**

- Splits the string into a list of substrings using ``sep`` as the delimiter.
- Default separator is whitespace if ``sep`` is None.
- ``maxsplit`` limits the number of splits performed; remaining string is kept as last element.
- Returns a list of substrings.
- Examples include splitting on space, letters, commas, or hyphens.
- If the separator is not found, the returned list contains the original string as its only element.

- **rsplit(sep=None, maxsplit=-1)**

- Works like split() but starts splitting from the right (end) of the string.
- Useful when you want to split a fixed number of times from the end.

- **splitlines(keepends=False)**

- Splits a multi-line string at line boundaries into a list of lines.
- Returns each line as a separate string.

- `keepends` parameter (not covered in detail) can optionally keep newline characters in the resulting list.

- Useful for processing multi-line text.

Section 3: Example Code and Use Cases

replace() method examples

Replace all hyphens with commas

Replace only first three hyphens with commas

Replace a substring not found in string (no change)

Replace domain in email string

```
s1 = 'a-b-c-d-e'
s2 = s1.replace('-', ',')
print(s2) # Output: a,b,c,d,e

s2 = s1.replace('-', ', ', 3)
print(s2) # Output: a,b,c,d-e

s2 = s1.replace('k', 'm')
print(s2) # Output: a-b-c-d-e

email = 'abcd@gmail.com'
new_email = email.replace('gmail', 'yahoo')
print(new_email) # Output: abcd@yahoo.com
```

Explanation: Demonstrates replacing characters and substrings with optional count limiting, and illustrates that no change occurs if target substring is absent.

join() method examples

Insert s1 string between characters of s2

Another example using '/'

```
s1 = 'xyz'
s2 = 'ABC'
```

```
s3 = s1.join(s2)
print(s3) # Output: AxyzBxyzC

separator = '/'
s3 = separator.join(s2)
print(s3) # Output: A/B/C
```

Explanation: Shows how join inserts the separator string between iterable elements in a new string.

split() method examples

Split by default separator (space)

Split by character 'h'

Split by comma

Split by hyphen with maxsplit limiting splits

```
s1 = 'John Smith Ajay'

s2 = s1.split()
print(s2) # Output: ['John', 'Smith', 'Ajay']

s2 = s1.split('h')
print(s2) # Output: ['Jo', 'n Smit', ' Ajay']

s3 = 'John,Smith,Ajay'
s2 = s3.split(',')
print(s2) # Output: ['John', 'Smith', 'Ajay']

s4 = 'John-Smith-Ajay-khan-james'
s2 = s4.split('-', 3)
print(s2) # Output: ['John', 'Smith', 'Ajay', 'khan-james']
```

Explanation: Shows splitting by various delimiters and with limit on number of splits, producing lists of substrings.

rsplit() example

Split 3 times from right

```
s = 'John-Smith-Ajay-khan-james'

s2 = s.rsplit('-', 3)
print(s2) # Output: ['John', 'Smith', 'Ajay', 'khan', 'james']
```

Explanation: Demonstrates splitting from the right side, useful for processing suffixes.

splitlines() example

```
multiline_str = "Line 1\nLine 2\nLine 3"
lines = multiline_str.splitlines()
print(lines) # Output: ['Line 1', 'Line 2', 'Line 3']
```

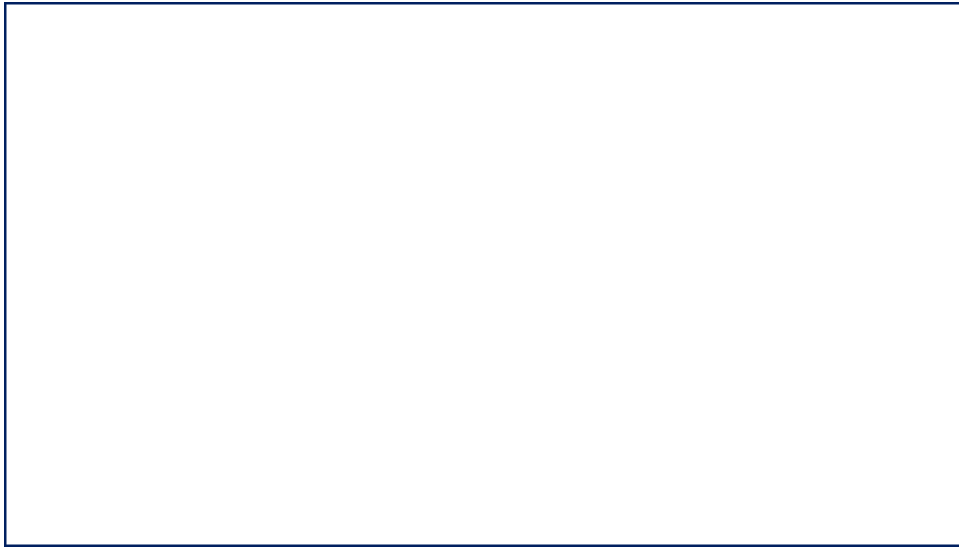
Explanation: Splits a multi-line string into individual lines.

Section 4: Key Takeaways

- String methods like `replace()`, `join()`, `split()`, `rsplit()`, and `splitlines()` are powerful tools for text manipulation in Python.
- All these methods return new strings or lists; they do not modify the original string due to string immutability.
- The `replace()` method can replace all or a limited count of occurrences of a substring.
- The `join()` method concatenates a sequence inserting the separator string only between elements.
- The `split()` method divides strings on a delimiter into lists, with optional limits on splits.
- `rsplit()` does split operations starting from the right side of the string.
- `splitlines()` efficiently splits multi-line text into a list of lines.
- Understanding and practicing these methods is essential for effective string handling in Python programming and application development.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Join & Split



Join & Split
String Methods

Join & Split

String Methods

Join & Split

String Methods

replace(old, new, count)

Join & Split

String Methods

replace(old, new, count)

S1

a	-	b	-	c	-	d	-	e
---	---	---	---	---	---	---	---	---

Join & Split

String Methods

replace(old, new, count)

join(iterable)

S1

a	b	c
---	---	---

Join & Split

String Methods

replace(old, new, count)

join(iterable)

/

 s1

a

b

c

Join & Split

String Methods

replace(old, new, count)

join(iterable)

/

 s1

a

/

b

/

c

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

p	y	t	h	o	n		i	s		e	a	s	y
---	---	---	---	---	---	--	---	---	--	---	---	---	---

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

p	y	t	h	o	n		i	s		e	a	s	y
---	---	---	---	---	---	--	---	---	--	---	---	---	---

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

Join & Split

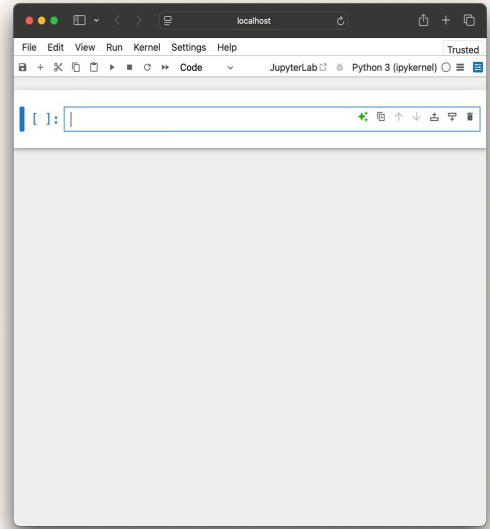
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



Join & Split

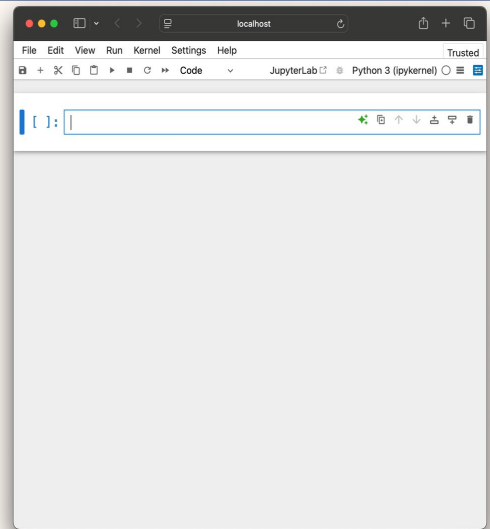
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



Join & Split

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
[1]: S1 = 'a-b-c-d-e'
```

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e

```
S2 = S1.replace('-', ',')
```

Join & Split

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
[1]: S1 = 'a-b-c-d-e'
```

```
[2]: S2 = S1.replace('-', ',')
```

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e

```
S2 = S1.replace('-', ',')
```

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
localhost
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('-', ',')

{ }:
```

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e
S2	a	,	b	,	c	,	d	,	e

S2 = S1.replace('-', ',')

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
localhost
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('-', ',')
[3]: print(S2)
a,b,c,d,e

{ }:
```

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e
S2	a	,	b	,	c	,	d	,	e

S2 = S1.replace('-', ',')

Join & Split

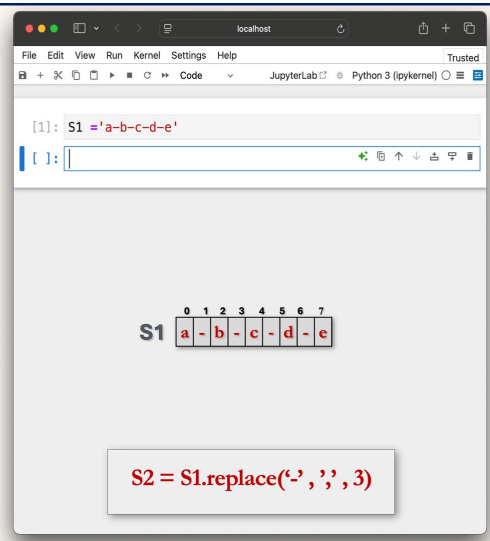
replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)



The screenshot shows a JupyterLab window with a code cell containing the following Python code:

```
[1]: S1 = 'a-b-c-d-e'
```

The output of the code cell is a string representation of the variable S1, which is 'a-b-c-d-e'. Below the output, there is a visual representation of the string as a sequence of characters in a table:

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e

Below the table, there is a code cell containing the following Python code:

```
S2 = S1.replace('-', '', 3)
```

Join & Split

Using Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
```

```
[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('-',',',3)
```

S1

0	1	2	3	4	5	6	7	
a	-	b	-	c	-	d	-	e

S2 = S1.replace('-',',',3)

Join & Split

Using Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
```

```
[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('-',',',3)
[ ]:
```

S1

0	1	2	3	4	5	6	7	
a	-	b	-	c	-	d	-	e

S2

0	1	2	3	4	5	6	7	
a	,	b	,	c	,	d	-	e

S2 = S1.replace('-',',',3)

Join & Split

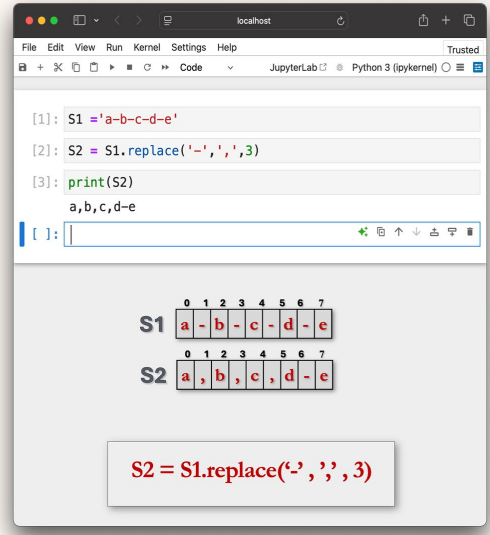
replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)



The screenshot shows a JupyterLab window with a code cell containing the following Python code:

```
[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('-', ', ', 3)
[3]: print(S2)
```

The output of the code cell is:

```
a,b,c,d-e
```

Below the code cell, there is a visual representation of the string S1 and its transformation into S2. S1 is shown as a sequence of characters 'a', 'b', 'c', 'd', 'e' with indices 0 through 7. S2 is shown as a sequence of characters 'a', 'b', 'c', 'd', 'e' with indices 0 through 7. The transformation is S2 = S1.replace('-', ', ', 3).

Join & Split

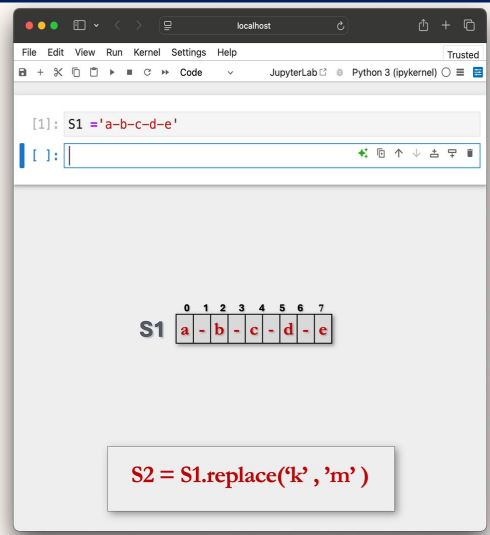
replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)



The screenshot shows a JupyterLab window with a code cell containing the following Python code:

```
[1]: S1 = 'a-b-c-d-e'
```

The output of the code cell is:

```
a,b,c,d-e
```

Below the code cell, there is a visual representation of the string S1 and its transformation into S2. S1 is shown as a sequence of characters 'a', 'b', 'c', 'd', 'e' with indices 0 through 7. S2 is shown as a sequence of characters 'a', 'b', 'c', 'd', 'e' with indices 0 through 7. The transformation is S2 = S1.replace('-', ', ', 3).

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('k', 'm')
[ ]:
```

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e
S2	a	-	b	-	c	-	d	-	e

S2 = S1.replace('k', 'm')

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
[1]: S1 = 'a-b-c-d-e'
[2]: S2 = S1.replace('k', 'm')
[3]: print(S2)
a-b-c-d-e
[ ]:
```

	0	1	2	3	4	5	6	7	
S1	a	-	b	-	c	-	d	-	e
S2	a	-	b	-	c	-	d	-	e

S2 = S1.replace('k', 'm')

Join & Split

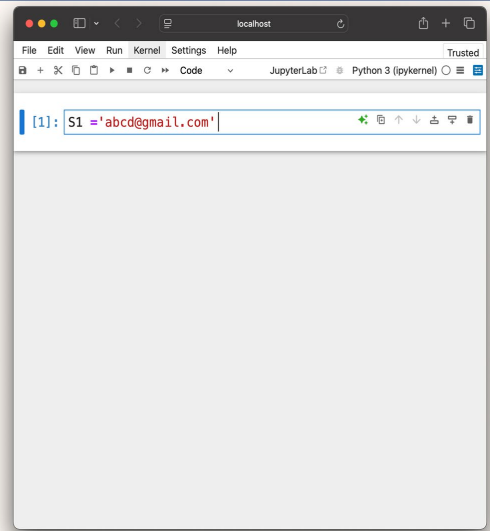
replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)



```
[1]: S1 = 'abcd@gmail.com'
```

Join & Split

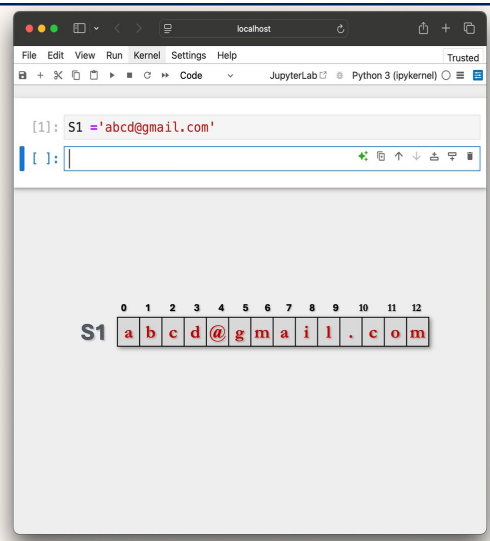
replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)



```
[1]: S1 = 'abcd@gmail.com'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	a	b	c	d	@	g	m	a	i	l	.	c	o	m

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
localhost
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: S1 = 'abcd@gmail.com'
{}:
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	a	b	c	d	@	g	m	a	i	l	.	c	o	m

```
S2 = S1.replace('gmail', 'yahoo')
```

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
localhost
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: S1 = 'abcd@gmail.com'
{}:
```

```
[2]: S2 = S1.replace('gmail', 'yahoo')
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	a	b	c	d	@	g	m	a	i	l	.	c	o	m

```
S2 = S1.replace('gmail', 'yahoo')
```

Join & Split

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
[1]: S1 = 'abcd@gmail.com'
[2]: S2 = S1.replace('gmail','yahoo')
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	a	b	c	d	@	g	m	a	i	l	.	c	o	m
S2	a	b	c	d	@	y	a	h	o	o	.	c	o	m

S2 = S1.replace('gmail', 'yahoo')

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: S1 = 'abcd@gmail.com'
[2]: S2 = S1.replace('gmail','yahoo')
[3]: print(S2)
abcd@yahoo.com

[ ]: |
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	a	b	c	d	@	g	m	a	i	l	.	c	o	m
S2	a	b	c	d	@	y	a	h	o	o	.	c	o	m

S2 = S1.replace('gmail', 'yahoo')

Join & Split

String Methods

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[ ]: |
```

Join & Split

Joining Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'xyz'
```

	0	1	2
S1	x	y	z

Join & Split

Joining Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'xyz'
```

```
[2]: S2 = 'abc'
```

	0	1	2
S1	x	y	z

	0	1	2
S2	a	b	c

Join & Split

String Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'xyz'
```

```
[2]: S2 = 'abc'
```

```
[ ]:
```

	0	1	2
S1	x	y	z
S2	a	b	c

`S3 = S1.join(S2)`

Join & Split

String Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'xyz'
```

```
[2]: S2 = 'abc'
```

```
[ ]: S3 = S1.join(S2)
```

	0	1	2
S1	x	y	z
S2	a	b	c

`S3 = S1.join(S2)`

Join & Split

Learning Objectives

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: S1 = 'xyz'
[2]: S2 = 'abc'
[3]: S3 = S1.join(S2)
```

S1

0

1

2

x

y

z

S2

0

1

2

a

b

c

S3

0

1

2

3

4

5

6

7

8

a

x

y

z

b

x

y

z

c

S3 = S1.join(S2)

Join & Split

Learning Objectives

replace(old, new, count)

join(iterable)

split(sep, maxsplit)

rsplit(sep, maxsplit)

splitlines(keepends)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: S1 = 'xyz'
[2]: S2 = 'abc'
[3]: S3 = S1.join(S2)
[4]: print(S3)
```

axyzbxyzc

S1

0

1

2

x

y

z

S2

0

1

2

a

b

c

S3

0

1

2

3

4

5

6

7

8

a

x

y

z

b

x

y

z

c

S3 = S1.join(S2)

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

The screenshot shows a JupyterLab window with a code cell containing the assignment `S1 = '/'`. Below the code cell, the variable inspector displays the variable `S1` with its value `'/'` at index 0.

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

The screenshot shows a JupyterLab window with two code cells. The first cell contains `S1 = '/'` and the second cell contains `S2 = 'abc'`. Below the code cells, the variable inspector displays the variables `S1` and `S2`. `S1` has the value `'/'` at index 0. `S2` has the value `'abc'` at indices 0, 1, and 2, with the characters `a`, `b`, and `c` displayed in separate boxes.

Join & Split

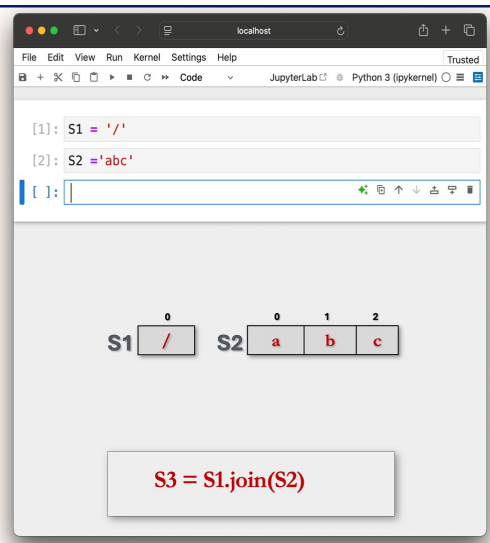
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



```
[1]: S1 = '/'
[2]: S2 = 'abc'
[ ]:
```

S1

/

 S2

a	b	c
---	---	---

S3 = S1.join(S2)

Join & Split

Replacing Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = '/'
[2]: S2 = 'abc'
[3]: S3 = S1.join(S2)
```

S1	0	/				
S2	0	a	1	b	2	c

S3 = S1.join(S2)

Join & Split

Replacing Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = '/'
[2]: S2 = 'abc'
[3]: S3 = S1.join(S2)
[ ]:
```

S1	0	/								
S2	0	a	1	b	2	c				
S3	0	a	1	/	2	b	3	/	4	c

S3 = S1.join(S2)

Join & Split

Joining Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = '/'
[2]: S2 = 'abc'
[3]: S3 = S1.join(S2)
[4]: print(S3)
a/b/c
```

`[ ]:`

S1:

0	/
---	---

S2:

0	a	1	b	2	c
---	---	---	---	---	---

S3:

0	a	1	/	2	b	3	/	4	c
---	---	---	---	---	---	---	---	---	---

`S3 = S1.join(S2)`

Join & Split

Joining Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[ ]:
```

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

The screenshot shows a JupyterLab window with a code editor and an output area. The code editor contains the line `S1 = 'John Smith Ajay'`. The output area displays the string `S1` followed by a table of character indices. The table has 13 columns, indexed from 0 to 12. The characters are: J (0), o (1), h (2), n (3), S (4), m (5), i (6), t (7), h (8), A (9), j (10), a (11), y (12).

0	1	2	3	4	5	6	7	8	9	10	11	12
J	o	h	n	S	m	i	t	h	A	j	a	y

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

The screenshot shows a JupyterLab window with a code editor and an output area. The code editor contains the line `S1 = 'John Smith Ajay'`. The output area displays the string `S1` followed by a table of character indices, identical to the one in the first image. Below the table, there is a code cell containing the line `S2 = S1.split()`.

0	1	2	3	4	5	6	7	8	9	10	11	12
J	o	h	n	S	m	i	t	h	A	j	a	y

```
S2 = S1.split()
```

Join & Split

Learning Objectives

- `replace( old, new, count )`
- `join( iterable )`
- `split( sep, maxsplit )`**
- `rsplit( sep, maxsplit )`
- `splitlines( keepends )`

```
[1]: S1 = 'John Smith Ajay'
```

```
[2]: S2 = S1.split()
```

0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	J	o	h	n	S	m	i	t	h	A	j	a	y

`S2 = S1.split()`

Join & Split

Learning Objectives

- `replace( old, new, count )`
- `join( iterable )`
- `split( sep, maxsplit )`**
- `rsplit( sep, maxsplit )`
- `splitlines( keepends )`

```
[1]: S1 = 'John Smith Ajay'
```

```
[2]: S2 = S1.split()
```

```
[ ]:
```

0	1	2	3	4	5	6	7	8	9	10	11	12	
S1	J	o	h	n	S	m	i	t	h	A	j	a	y

0	1	2	
S2	J o h n	S m i t h	A j a y

`S2 = S1.split()`

Join & Split

String Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John Smith Ajay'
[2]: S2 = S1.split()
[3]: print(S2)
['John', 'Smith', 'Ajay']
```

[]:

0	1	2	3	4	5	6	7	8	9	10	11	12					
S1			J	o	h	n		S	m	i	t	h		A	j	a	y
S2			0			1			2								
			J	o	h	n		S	m	i	t	h		A	j	a	y

`S2 = S1.split()`

Join & Split

String Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John Smith Ajay'
[2]: S2 = S1.split('h')
```

0	1	2	3	4	5	6	7	8	9	10	11	12					
S1			J	o	h	n		S	m	i	t	h		A	j	a	y

`S2 = S1.split('h')`

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

The screenshot shows a JupyterLab window with the following content:

```
[1]: S1 = 'John Smith Ajay'
[6]: S2 = S1.split('h')
```

Below the code, a visual representation of the string splitting is shown:

S1

0	1	2	3	4	5	6	7	8	9	10	11	12		
J	o	h	n		S	m	i	t	h		A	j	a	y

S2

0	1	2											
J	o		n		S	m	i	t		A	j	a	y

S2 = S1.split('h')

Join & Split

String Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John Smith Ajay'
[2]: S2 = S1.split('h')
[8]: print(S2)
['Jo', 'n Smit', ' Ajay']
```

S2 = S1.split('h')

Join & Split

String Methods

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John,Smith,Ajay'
```

S2 = S1.split(',')

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John,Smith,Ajay'
[2]: S2 = S1.split(',')
[]:
```

Diagram illustrating string splitting:

S1: J o h n , S m i t h , A j a y (indices 0 to 12)

S2: J o h n S m i t h A j a y (indices 0 to 2)

`S2 = S1.split(',')`

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John,Smith,Ajay'
[2]: S2 = S1.split(',')
[3]: print(S2)
['John', 'Smith', 'Ajay']
[]:
```

Diagram illustrating string splitting:

S1: J o h n , S m i t h , A j a y (indices 0 to 12)

S2: J o h n S m i t h A j a y (indices 0 to 2)

`S2 = S1.split(',')`

Join & Split

String Methods

```
replace( old, new, count )  
join( iterable )  
split( sep, maxsplit )  
rsplit( sep, maxsplit )  
splitlines( keepends )
```

The screenshot shows a JupyterLab window with the following content:

```
[1]: S1 = 'John-Smith-Ajay-khan-james'
```

Below the code cell, the string S1 is visualized as a list of characters, with each character in a separate box and its index (0-25) above it:

Index	Character
0	J
1	o
2	h
3	n
4	-
5	S
6	m
7	i
8	t
9	h
10	-
11	A
12	j
13	a
14	y
15	-
16	K
17	h
18	a
19	n
20	-
21	J
22	a
23	m
24	e
25	s

Join & Split

String Methods

```
replace( old, new, count )  
join( iterable )  
split( sep, maxsplit )  
rsplit( sep, maxsplit )  
splitlines( keepends )
```

The screenshot shows a JupyterLab window with the following content:

```
[1]: S1 = 'John-Smith-Ajay-khan-james'
```

Below the code cell, the string S1 is visualized as a list of characters, with each character in a separate box and its index (0-25) above it:

Index	Character
0	J
1	o
2	h
3	n
4	-
5	S
6	m
7	i
8	t
9	h
10	-
11	A
12	j
13	a
14	y
15	-
16	K
17	h
18	a
19	n
20	-
21	J
22	a
23	m
24	e
25	s

```
S2 = S1.split()
```

```
replace( old, new, count )
```

```
join( iterable )
```

split(sep, maxsplit)

```
rsplit( sep, maxsplit )
```

```
splitlines( keepends )
```

S2 = S1.split()

```
replace( old, new, count )
```

```
join( iterable )
```

```
split( sep, maxsplit )
```

```
rsplit( sep, maxsplit )
```

```
splitlines( keepends )
```

S2 = S1.split()

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John-Smith-Ajay-khan-james'
[2]: S2 = S1.split()
[3]: print(S2)
['John-Smith-Ajay-khan-james']
[ ]:
```

S1 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 = S1.split()

Join & Split

`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`

```
[1]: S1 = 'John-Smith-Ajay-khan-james'
[2]: S2 = S1.split('-')

```

S1 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 = S1.split('-')

Join & Split

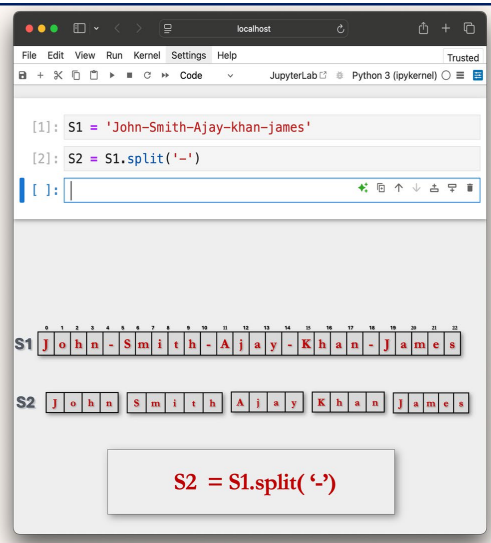
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



The screenshot shows a JupyterLab window with a Python 3 (ipykernel) environment. The code cell contains two lines of Python code:

```
[1]: S1 = 'John-Smith-Ajay-khan-james'
[2]: S2 = S1.split('-')
```

Below the code cell, the output is displayed. It shows the string `S1` as `'John-Smith-Ajay-khan-james'` and the list `S2` as `['John', 'Smith', 'Ajay', 'khan', 'james']`. The output is visualized as a grid of characters for each string, with the hyphen character highlighted in red in the original image. Below the output, a button displays the code `S2 = S1.split('-')`.

Join & Split

String Methods

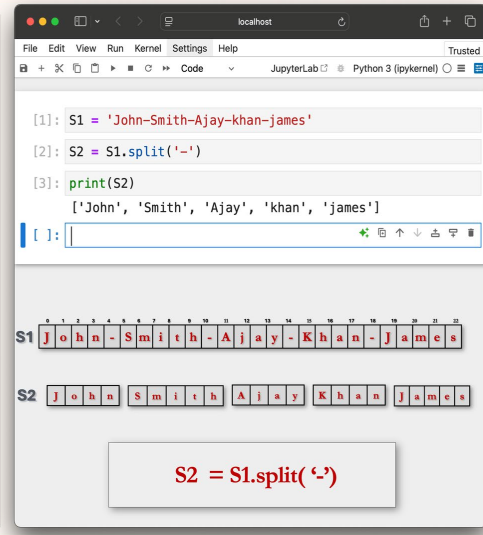
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



```
[1]: S1 = 'John-Smith-Ajay-khan-james'
[2]: S2 = S1.split('-')
[3]: print(S2)
['John', 'Smith', 'Ajay', 'khan', 'james']
```

S1 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 J o h n S m i t h A j a y K h a n J a m e s

S2 = S1.split('-')

Join & Split

String Methods

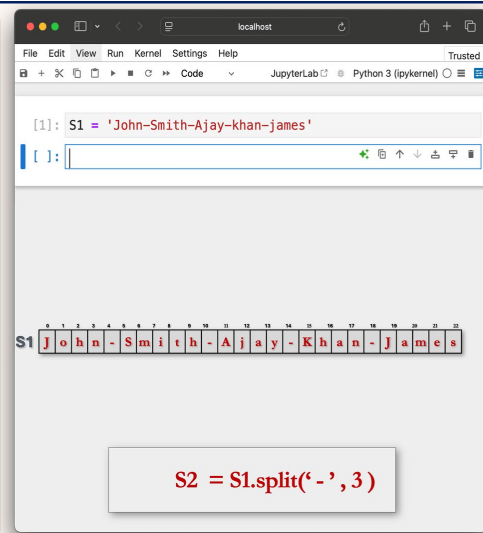
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



```
[1]: S1 = 'John-Smith-Ajay-khan-james'
[2]: S2 = S1.split('-', 3)
```

S1 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 J o h n S m i t h A j a y K h a n - J a m e s

S2 = S1.split('-', 3)

Join & Split

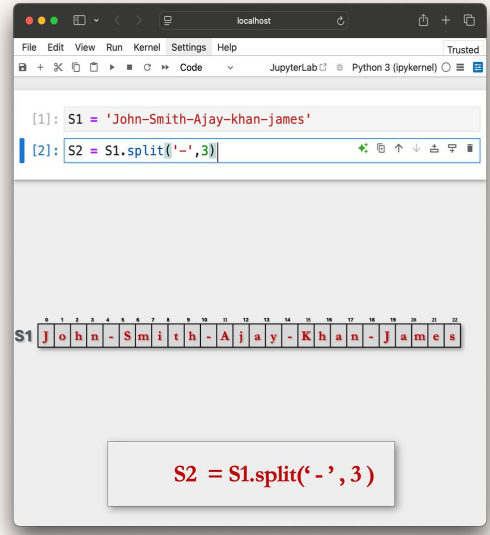
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



```
[1]: S1 = 'John-Smith-Ajay-khan-james'
```

```
[2]: S2 = S1.split('-',3)
```

S1 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 = S1.split('-', 3)

Join & Split

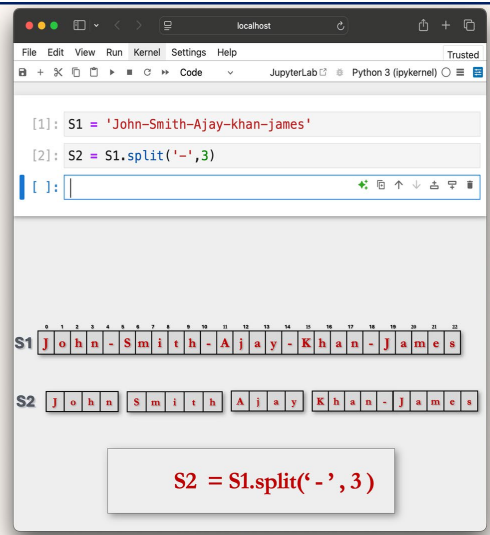
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



```
[1]: S1 = 'John-Smith-Ajay-khan-james'
```

```
[2]: S2 = S1.split('-',3)
```

```
[ ]:
```

S1 J o h n - S m i t h - A j a y - K h a n - J a m e s

S2 J o h n S m i t h A j a y K h a n J a m e s

S2 = S1.split('-', 3)

Join & Split

String Methods

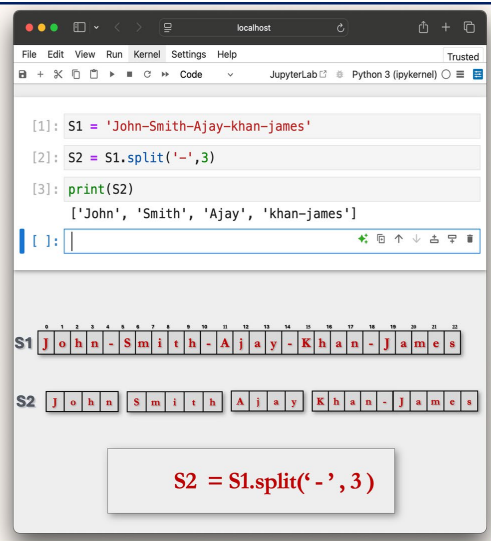
`replace( old, new, count )`

`join( iterable )`

`split( sep, maxsplit )`

`rsplit( sep, maxsplit )`

`splitlines( keepends )`



The screenshot shows a JupyterLab window with the following code and output:

```
[1]: S1 = 'John-Smith-Ajay-khan-james'
```

```
[2]: S2 = S1.split('-',3)
```

```
[3]: print(S2)
```

```
['John', 'Smith', 'Ajay', 'khan-james']
```

The output is displayed as a list of four strings: `['John', 'Smith', 'Ajay', 'khan-james']`. Below the code, there are two visual representations of the strings:

S1 John-Smith-Ajay-khan-james

S2 John Smith Ajay Khan-james

At the bottom, a box contains the code: `S2 = S1.split('-', 3)`